Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

**Ю.Е. Гапанюк**

# МЕТОДИЧЕСКИЕ УКАЗАНИЯ

## по дисциплине «Архитектура автоматизированных систем обработки информации и управления» Профиль: «Архитектура программных систем»

## Учебные материалы домашнего задания №3 Разработка приложения на языке C# с базой данных и пользовательским интерфейсом

Москва – 2026

# Оглавление

**1 ВВЕДЕНИЕ ..... 3**

**2 ТРЕБОВАНИЯ К РАЗРАБАТЫВАЕМОМУ ПРИЛОЖЕНИЮ ..... 3**

2.1 Технологический стек ..... 3

2.2 Что должно быть разработано ..... 3

**3 ВАРИАНТЫ ЗАДАНИЙ ..... 5**

**4 ЧАСТЬ 1. МОДЕЛЬ ДАННЫХ И DBCONTEXT ..... 8**

4.1 Классы-модели (Entity-классы) ..... 8

4.2 Контекст базы данных ..... 8

4.3 Создание базы данных и начальные данные ..... 9

**5 ЧАСТЬ 2. ФОРМА (ИЛИ КОНТРОЛЛЕР) ДЛЯ СПРАВОЧНОЙ ТАБЛИЦЫ (MASTER) ..... 9**

**6 ЧАСТЬ 3. ФОРМА (ИЛИ КОНТРОЛЛЕР) ДЛЯ ОСНОВНОЙ ТАБЛИЦЫ (DETAIL) ..... 10**

**7 ЧАСТЬ 4. ОТЧЕТ ..... 11**

**8 ЧАСТЬ 5. НАВИГАЦИЯ МЕЖДУ ФОРМАМИ (КОНТРОЛЛЕРАМИ) ..... 12**

# 1 Введение

В домашнем задании №2 Вы реализовали консольное приложение с прямым обращением к SQLite через ADO.NET и паттерн Fluent Interface для отчётов. В этом задании мы переведем приложение на современный стек: Entity Framework Core вместо прямых SQL-запросов и графический или веб-интерфейс вместо консольного меню.

По умолчанию варианты предметных областей остаются теми же, что и в домашнем задании №2 (по номеру в списке группы). По желанию предметную области можно поменять.

# 2 Требования к разрабатываемому приложению

## *2.1 Технологический стек*

- Язык программирования: C#
- База данных: SQLite
- Доступ к данным: Entity Framework Core (Code First); прямые SQL-строки не используются
- Запросы к данным: LINQ (LINQ to Entities)
- Интерфейс пользователя (на выбор): Windows Forms, WPF или ASP.NET MVC (или что-то иное по желанию).

## *2.2 Что должно быть разработано*

**1. Модель данных**

- Класс-модель для справочной таблицы (master) с навигационным свойством-коллекцией.
- Класс-модель для основной таблицы (detail) с навигационным свойством-ссылкой на справочник.
- Класс AppDbContext, наследующий от DbContext, с двумя DbSet<T>
- Автоматическое создание БД при первом запуске (EnsureCreated)

- Заполнение начальными данными при пустой БД (не менее 4 записей в справочнике, не менее 12 в основной таблице).

**2. Форма / контроллер справочной таблицы (master)**

- Просмотр — список всех записей справочника
- Добавление — форма/страница ввода новой записи
- Редактирование — форма/страница изменения существующей записи
- Удаление — с подтверждением; запрет удаления, если есть связанные записи в основной таблице

**3. Форма / контроллер основной таблицы (detail)**

- Просмотр — список всех записей с отображением названия категории из справочника (через Include).
- Добавление — форма/страница; категория выбирается из выпадающего списка.
- Редактирование — форма/страница; категория выбирается из выпадающего списка.
- Удаление — с подтверждением.
- Валидация числового поля: значение не может быть отрицательным.

**4. Форма / контроллер отчёта**

- Раздел 1. Полный список записей основной таблицы с названием категории (LINQ: Include + OrderBy)
- Раздел 2. Количество записей по категориям (LINQ: GroupBy + Count)
- Раздел 3. Среднее значение числового поля по категориям, сортировка по убыванию (LINQ: GroupBy + Average + OrderByDescending)

**5. Навигация**

Главное окно / меню / навигационная панель с переходом ко всем трём разделам приложения.

**6. Требования к качеству кода**

Все публичные классы и их публичные члены снабжены XML-комментариями.

Объекты AppDbContext создаются через конструкцию using.

Ошибки ввода обрабатываются: пользователю выводится понятное сообщение, приложение не завершается аварийно.

Прямые SQL-строки (FromSqlRaw, ExecuteSqlRaw) не используются.

## 3 Варианты заданий

Номер варианта — по номеру в списке группы. Структура для всех вариантов для связи двух таблиц один-ко-многим одинаковая:

- Справочная таблица (сторона «один»): идентификатор и название.
- Основная таблица (сторона «много»): идентификатор, внешний ключ на справочник, название и числовой показатель.

Для сравнения — семинарский пример: справочник `dep(dep_id, dep_name)`, основная таблица `dev(dev_id, dep_id, dev_name, dev_commits)`.

| № | Справочник | Основная таблица | Числовое поле | Расшифровка |
|---|---|---|---|---|
| 1 | Жанры | Книги | `pages` | Количество страниц |
| 2 | Киностудии | Фильмы | `budget_mln` | Бюджет фильма (млн долл.) |
| 3 | Автомобильные марки | Автомобили | `horsepower` | Мощность двигателя (л.с.) |
| 4 | Факультеты | Студенты | `gpa` | Средний балл успеваемости |
| 5 | Категории товаров | Товары | `price` | Цена (руб.) |

| | | | | |
|---|---|---|---|---|
| 6 | Должности | Работники | `salary` | Зарплата (тыс. руб./мес.) |
| 7 | Музыкальные альбомы | Песни | `duration_sec` | Длительность трека (сек.) |
| 8 | Кухни мира | Рецепты | `cook_min` | Время приготовления (мин.) |
| 9 | Страны | Города | `population_k` | Население (тыс. чел.) |
| 10 | Лаборатории | Эксперименты | `duration_hours` | Длительность опыта (часов) |
| 11 | Производители электроники | Смартфоны | `price` | Цена (руб.) |
| 12 | Игровые платформы | Видеоигры | `rating` | Оценка игроков (баллы из 100) |
| 13 | Музеи | Экспонаты | `value_k` | Оценочная стоимость (тыс. руб.) |
| 14 | Авиакомпании | Рейсы | `distance_km` | Дальность маршрута (км) |
| 15 | Футбольные клубы | Футболисты | `goals` | Количество забитых голов |
| 16 | Больницы | Врачи | `experience` | Стаж работы (лет) |
| 17 | Издательства | Журналы | `circulation_k` | Тираж (тыс. экземпляров) |
| 18 | Кафедры | Преподаватели | `publications` | Количество научных |

| | | | | |
|---|---|---|---|---|
| | | | | публикаций |
| 19 | Магазины | Заказы | `amount` | Сумма заказа (руб.) |
| 20 | Проекты | Задачи | `hours` | Трудоёмкость (человеко-часов) |
| 21 | Микрорайоны | Жилые дома | `floors` | Количество этажей |
| 22 | Рестораны | Блюда в меню | `price` | Цена блюда (руб.) |
| 23 | Теплицы | Растения | `height_cm` | Высота растения (см) |
| 24 | Операторы связи | Тарифы | `price_month` | Абонентская плата (руб./мес.) |
| 25 | Зоопарки | Животные | `weight_kg` | Масса животного (кг) |
| 26 | Школы | Ученики | `avg_grade` | Средний балл успеваемости |
| 27 | Серверы | Базы данных | `size_gb` | Объём данных (ГБ) |
| 28 | Склады | Партии товаров | `quantity` | Количество единиц (шт.) |
| 29 | Автопарки | Транспортные средства | `mileage_km` | Пробег (км) |
| 30 | Телеканалы | Телепередачи | `viewers_k` | Аудитория (тыс. зрителей) |

# 4 Часть 1. Модель данных и DbContext

## 4.1 Классы-модели (Entity-классы)

Создайте два класса-модели для Entity Framework Core — аналоги классов из ДЗ №2, адаптированные для работы с EF Core.

Требования к классам:

- Свойства (properties) для всех столбцов таблицы.
- Навигационные свойства для отражения связи «один-ко-многим»: у класса справочника — коллекция ICollection<T>, у класса основной таблицы — ссылка на объект справочника.
- XML-комментарии для класса и его публичных членов.

**Пример:**

```
/// <summary>Отдел (справочная таблица, сторона «один»)</summary>
public class Department
{
    /// <summary>Идентификатор отдела (первичный ключ)</summary>
    public int Id { get; set; }

    /// <summary>Название отдела</summary>
    public string Name { get; set; } = "";

    /// <summary>Навигационное свойство: разработчики этого отдела</summary>
    public ICollection<Developer> Developers { get; set; } = new List<Developer>();
}

/// <summary>Разработчик (основная таблица, сторона «много»)</summary>
public class Developer
{
    /// <summary>Идентификатор разработчика (первичный ключ)</summary>
    public int Id { get; set; }

    /// <summary>Идентификатор отдела (внешний ключ)</summary>
    public int DepartmentId { get; set; }

    /// <summary>Название отдела (навигационное свойство)</summary>
    public Department? Department { get; set; }

    /// <summary>Имя разработчика</summary>
    public string Name { get; set; } = "";

    /// <summary>Количество коммитов</summary>
    public int Commits { get; set; }
}
```

## 4.2 Контекст базы данных

Создайте класс AppDbContext, наследующий от DbContext:

```
public class AppDbContext : DbContext
{
    public DbSet<Department> Departments { get; set; }
    public DbSet<Developer> Developers { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder options)
        => options.UseSqlite("Data Source=app.db");
}
```

### *4.3 Создание базы данных и начальные данные*

- При первом запуске приложение должно создать базу данных с помощью context.Database.EnsureCreated().
- Если таблицы пусты — заполните их начальными данными: не менее 4 записей в справочнике и не менее 12 записей в основной таблице.
- Начальные данные можно задать в коде или загрузить из CSV-файлов ДЗ №2.

## 5 Часть 2. Форма (или контроллер) для справочной таблицы (master)

Реализуйте форму (в WinForms/WPF) или контроллер с представлениями (в ASP.NET MVC) для работы со справочной таблицей.

Требуемая функциональность (CRUD):

| Операция | Описание |
|---|---|
| Просмотр | Отображение списка всех записей справочника |
| Добавление | Форма/страница для ввода названия новой записи; сохранение в БД |
| Редактирование | Форма/страница для изменения названия существующей записи |
| Удаление | Удаление записи с подтверждением; если с записью связаны записи основной таблицы — вывести предупреждение и запретить удаление |

**Требования к реализации:**

- Все операции с базой данных выполняются через LINQ to Entities (методы DbSet: Add, Update, Remove, Find, ToList и др.).
- Прямые SQL-запросы (FromSqlRaw, ExecuteSqlRaw) не используются.
- При ошибках ввода (пустое название и т.п.) отображается понятное сообщение пользователю.

Пример LINQ-запроса для получения всех отделов:

```
using var context = new AppDbContext();
var departments = context.Departments.OrderBy(d => d.Name).ToList();
```

# 6 Часть 3. Форма (или контроллер) для основной таблицы (detail)

Реализуйте форму (в WinForms/WPF) или контроллер с представлениями (в ASP.NET MVC) для работы с основной таблицей.

Требуемая функциональность (CRUD):

| Операция | Описание |
|---|---|
| Просмотр | Отображение списка всех записей с названием связанной категории из справочника (не просто ID) |
| Добавление | Форма/страница с полями для всех атрибутов; категория выбирается из выпадающего списка, загруженного из справочника |
| Редактирование | Форма/страница для изменения всех полей записи; категория выбирается из выпадающего списка |
| Удаление | Удаление записи с подтверждением |

**Требования к реализации:**

- Числовое поле должно проходить валидацию: значение не может быть отрицательным. При некорректном вводе необходимо выводить сообщение об ошибке.
- При добавлении и редактировании выпадающий список категорий формируется LINQ-запросом к справочнику.

- Список записей основной таблицы загружается с подгрузкой связанных данных (Include), чтобы отображать название категории. Пример:

```
using var context = new AppDbContext();
    var developers = context.Developers
        .Include(d => d.Department)
        .OrderBy(d => d.Name)
        .ToList();
```

## 7 Часть 4. Отчет

Реализуйте отдельную форму (в WinForms/WPF) или отдельное представление/контроллер (в ASP.NET MVC) для вывода отчёта.

**Требования к отчёту:**

Отчёт формируется исключительно средствами LINQ (без написания SQL-строк вручную) и содержит три раздела:

**Раздел 1. Полный список — все записи основной таблицы с названием категории из справочника, отсортированные по названию.**

Пример LINQ-запроса:

```
var report1 = context.Developers
.Include(d => d.Department)
.OrderBy(d => d.Name)
.Select(d => new {
    d.Name,
    DepartmentName = d.Department!.Name,
    d.Commits
})
.ToList();
```

**Раздел 2. Количество записей по категориям — группировка с агрегатной функцией Count().**

Пример:

```
var report2 = context.Developers
    .GroupBy(d => d.Department!.Name)
    .Select(g => new {
        Department = g.Key,
        Count = g.Count()
    })
    .OrderBy(r => r.Department)
    .ToList();
```

**Раздел 3. Среднее значение числового поля по категориям — группировка с агрегатной функцией Average(), сортировка по убыванию среднего значения.**

Пример:

```
var report3 = context.Developers
    .GroupBy(d => d.Department!.Name)
    .Select(g => new {
        Department = g.Key,
        AvgCommits = g.Average(d => d.Commits)
    })
    .OrderByDescending(r => r.AvgCommits)
    .ToList();
```

Все три раздела отображаются в одной форме/странице. Результаты рекомендуется выводить в виде таблиц (в WinForms — DataGridView, в WPF — DataGrid, в ASP.NET MVC — HTML-таблицы).

## 8 Часть 5. Навигация между формами (контроллерами)

- WinForms/WPF: реализуйте главное окно (или главное меню) с кнопками/пунктами для перехода к форме справочника, форме основной таблицы и форме отчёта.
- ASP.NET MVC: реализуйте навигационное меню в общем макете (_Layout), содержащее ссылки на контроллеры справочника, основной таблицы и отчёта.